

Tutorial: Automate your SP-API Calls Using a C# SDK

English

Home

Documentation

<> API Reference

Code Samples

Announcements

Models

Release Notes

FAQ

GitHub

Videos

GET STARTED

Welcome to SP-API Documentation

What is the Selling Partner API?

Selling Partner API Onboarding Overview



Terminology

Frequently Asked Questions



CHANGELOG

SP-API Release Notes

SP-API Deprecation Schedule

SP-API Product Metadata Updates

DIRECTORIES

Tutorial: Automate your SP-API Calls Using a C# SDK

Automate your SP-API calls with Login with Amazon (LWA) token exchange and authentication.

This tutorial provides you with all the required information to generate a C# SDK with Login with Amazon (LWA) token exchange and an authentication to build your application.

You can use this SDK to integrate Amazon marketplace features into your applications, including accessing product information, managing orders, handling shipments, and more.

Prerequisites

To complete this tutorial, you need the following prerequisites:

- A hybrid or SP-API app in draft or published state
- Integrated development environment (IDE) software (this walkthrough uses Visual Studio IDE on Windows OS)

Before your application can connect to the Selling Partner API, you must register it, and it must be authorized by a selling partner. If you do not have a hybrid or SP-API app, follow the steps to [register as a developer](#), [register your application](#), and [Authorizing Selling Partner API applications](#). Then, return to this tutorial.

Next, set up your workspace for the tutorial.

Step 1. Set up your workspace

1. On your local drive, create a directory for this project and name it **SwaggerToCL**.
2. Download the following tools.
 1. IDE software (this walkthrough uses [Visual Studio](#) IDE on Windows OS)
 2. [GNU Wget](#)
 3. [Java 8 or newer](#)
3. Run the following command to download the Swagger Code Generator:

Bash

```
wget https://repo1.maven.org/maven2/io/swagger/swagger-codegen-cli/2.4.13/swagger-codegen-cli-2.4.13.jar -O swagger-codegen-cli.jar
```

SP-API Models

SP-API Seller Use Cases

SP-API Vendor Use Cases

Seller Central URLs

Vendor Central URLs

REGISTRATION

SP-API Registration Overview

Developer Registration Request Status

Third-Party Provider Registration

Service Provider Registration and Role Approval Process Overview

Website Guidelines for Public Developers and Service Providers

User Permissions for Service Providers

Register your Application

Update your Application Information

Rotate your Application's LWA Credentials

View your Application Information and Credentials

Learn How Sellers Authorize Service Providers

Selling Partner API Roles

Policies and Agreements

About Facial Data

AUTHORIZATION

Authorize Applications

Renew Authorizations

Revoke Authorizations

Authorization Limits

Special Authorization Types

INTEGRATION

Building Robust Amazon SP-API Applications

Tutorial: Automate your SP-API Calls Using a C# SDK

4. Copy `swagger-codegen-cli.jar` into your local directory `C:\SwaggerToCL`.

5. In GitHub, go to <https://github.com/amzn/selling-partner-api-models/tree/main/models> and use the following command to clone the `selling-partner-api-models` repository to your local directory `C:\SwaggerToCL`.

Bash

```
git clone https://github.com/amzn/selling-partner-api-models
```

6. Navigate to the `selling-partner-api-models\models` folder in your local copy of the repository and copy a JSON file from the models subfolders into `C:\SwaggerToCL`. This tutorial uses the `Sellers.json` file.

7. In GitHub, go to <https://github.com/amzn/selling-partner-api-models/tree/main/clients/sellingpartner-api-aa-csharp> and download the `sellingpartner-api-aa-csharp` folder to your local computer.

This folder provides helper classes to generate an `access token` for Amazon Selling Partner APIs. It is intended for use with the Selling Partner API Client Libraries generated by [Swagger Codegen](#) using the RestSharp library. It can also be integrated into custom projects.

Now that you have completed the required setup, the next step is to generate the C# SDK with the authentication and authorization classes provided in the `sellingpartner-api-aa-csharp` folder.

Step 2. Generate a C# SDK with LWA token exchange and authentication

1. Open Visual Studio.

2. In the `sellingpartner-api-aa-csharp` folder, select the `SellingPartnerAPIAuthAndAuthCSharp.sln` file and build it using the **build** option in Visual Studio. This generates the `Amazon.SellingPartnerAPIAA.dll` assembly in the folder `sellingpartner-api-aa-csharp\src\Amazon.SellingPartnerAPIAA\bin\Debug\netstandard2.0`.

3. Open a terminal and run the following commands to generate the C# client library. The default package name for the generated client library is `Swagger.IO`. These commands generate the client libraries with their respective API names as the package name instead of `Swagger.IO`.

1. In `C:\SwaggerToCL`, create a JSON file named `csharpConfig.json`, open an editor and add the following code. For `packageName`, use the same name of the API you want to generate the client library for:

JSON

```
{"packageName":"SellingPartnerAPI.SellerAPI","targetFramework":"v4.7.2"}
```

Run this command to generate C# code with a customized package name:

Bash

```
java -jar C:\SwaggerToCL\swagger-codegen-cli.jar generate -i C:\SwaggerToCL\[name of model].json -l csharp -t [path to selling-partner-api-models\clients\sellingpartner-api-aa-csharp folder]\src\Amazon.SellingPartnerAPIAA\resources\swagger-codegen\templates\ -o C:\SwaggerToCL\[name of client library] -c C:\SwaggerToCL\csharpConfig.json
```

This command uses `Sellers.json` to generate C# code:

Bash

```
java -jar C:\SwaggerToCL\swagger-codegen-cli.jar generate -i
C:\SwaggerToCL\Sellers.json -l csharp -t
C:\SwaggerToCL\sellingpartner-api-aa-
csharp\src\Amazon.SellingPartnerAPIAA\resources\swagger-
codegen\templates\ -o C:\SwaggerToCL\Sellers_CsharpCL -c
C:\SwaggerToCL\csharpConfig.json
```

The SDK is created in `C:\SwaggerToCL\Sellers_CsharpCL`. The next step is to write the actual application and connect to the SP-API using the generated C# SDK.

Step 3. Connect to the Selling Partner API using the generated C# SDK

1. In Visual Studio IDE, navigate to your generated library **Sellers_CsharpCL** folder and open the `.sln` file. The `.sln` package file will have the name as provided in [Step 2. Generate a C# SDK with LWA token exchange and authentication](#). In this example, the package file name is `SellingPartnerAPI.SellerAPI.sln`.
2. Right-click the `SellingPartnerAPI.SellerAPI` file and select **Add > References**.
3. In the **References** window, choose **Browse**, navigate to `sellingpartner-api-aa-csharp/src/Amazon.SellingPartnerAPIAA/bin/Debug/netstandard2.0` and choose `Amazon.SellingPartnerAPIAA.dll`.
4. Repeat steps 1-3 for the `SellingPartnerAPI.SellerAPI.Test` file.
5. Navigate to your generated library **Sellers_CsharpCL** folder, right-click the `SellingPartnerAPI.SellerAPI` file, choose **Options > General**, and change the target framework to **.net Framework v4.7 and newer**.
6. Repeat step 5 for the `SellingPartnerAPI.SellerAPI.Test` file.
7. Right-click the `SellingPartnerAPI.SellerAPI` file and choose **Manage NuGet Packages**. Make sure you have the following package versions (If not, install the respective NuGet packages):
 1. `JsonSubTypes 1.2.0` or newer
 2. `Newtonsoft.json 12.0.3` or newer
 3. `RestSharp 106.12.0`
 4. `RateLimiter 2.2.0` or newer (this will also install `ComposableAsync.Core`)
8. Right-click the `SellingPartnerAPI.SellerAPI.Test` file and choose **Manage NuGet Packages**. Install the **Nunit 2.6.4** package.



Manual installation of NuGet packages

In some scenarios (for example, using IOS), the required versions in the NuGet packages appear, but you can encounter an error. In this scenario, you must manually add the packages from the package folder in the file system:

1. In Visual Studio IDE, navigate to your generated library **Sellers_CsharpCL** folder and open the `.sln` file.
2. Right-click the `SellingPartnerAPI.SellerAPI` file and select **Add > References**.

Delete an Application From Your Developer Account

CALLING THE SP-API

Configuration Details

Call Structure

Development Tools

Performance Management

SOLUTION PROVIDER PORTAL

Manage Users in Solution Provider Portal

Manage Your Business And Contact information in Solution Provider Portal

Update Your Developer Profile in Solution Provider Portal

Unify Your Developer Accounts

Verify Your Identity in Solution Provider Portal

API Usage Metrics in Solution Provider Portal

TUTORIALS

Tutorial: Subscribe to the ORDER_CHANGE Notification

Tutorial: Test Selling Partner API Endpoints

Tutorial: Automate your SP-API Calls Using a C# SDK

Tutorial: Automate your SP-API Calls Using a Java SDK

Tutorial: Automate your SP-API Calls Using a JavaScript SDK for Node.js

Tutorial: Automate your SP-API Calls Using a Python SDK

Tutorial: Automate your SP-API Calls using a prebuilt C# SDK

Tutorial: Automate your SP-API Calls using prebuilt Java SDK

Tutorial: Automate your SP-API Calls using a prebuilt JavaScript SDK

Tutorial: Automate your SP-API Calls using prebuilt PHP SDK

Tutorial: Automate your SP-API calls using a prebuilt Python SDK

Tutorial: Authorize Multiple Vendor Central Accounts with a Single SP-API Application

Tutorial: Retrieve Merchant Shipping Templates

Tutorial: Grant the SP-API Permission to an Amazon SQS Queue

Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier

TROUBLESHOOTING

Troubleshoot SP-API Errors

URL Encoding

Resolve Common HTTP and Authorization Error Codes

External Fulfillment Errors ➤Listings Items API Issues Troubleshooting ➤

SELLING PARTNER APPSTORE

What is the Selling Partner Appstore?

List Your App on the Selling Partner Appstore

Edit Your Appstore Listing

Check Listing Status

Appstore Ratings and Reviews

Press Releases and Promotions

SERVICE PROVIDER NETWORK

List Your Service

SECURITY AND COMPLIANCE

SP-API Security and Compliance Overview

Amazon Selling Partner API Guard Implementation Guide ➤

Tutorial: Automate your SP-API Calls Using a C# SDK

3. In the **References** window, choose **Browse**, navigate to

sellingpartner-api-aa
csharp/src/Amazon.SellingPartnerAPIAA/bin/Debug/netstandard 2.0

, choose the generated client library folder (for example, **Sellers_CsharpCL**), then choose **packages** and add the package for which you are facing the error.

9. In the **SellingPartnerAPI.SellerAPI.Test > Api** folder, open the**[Modelname]Tests.cs** file and add the following code:

C#

```
using System;
using System.IO;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.Linq;
using System.Reflection;
using RestSharp;
using SellingPartnerAPI.SellerAPI.Client;
using SellingPartnerAPI.SellerAPI.Api;
using SellingPartnerAPI.SellerAPI.Model;
using Amazon.SellingPartnerAPIAA;

namespace SellingPartnerAPI.SellerAPI.Test
{
    class SellersApiTests
    {
        public static void Main(string[] args)
        {
            try
            {
                LWAAuthorizationCredentials
                lwaAuthorizationCredentials = new LWAAuthorizationCredentials
                {
                    ClientId = "amzn1.application-oa2-
client.*****",
                    ClientSecret =
                    "*****",
                    RefreshToken =
                    "Atzr|*****",
                    Endpoint = new
                    Uri("https://api.amazon.com/auth/o2/token")
                };
                SellersApi sellersApi = new SellersApi.Builder()
                .SetLWAAuthorizationCredentials(lwaAuthorizationCredentials)
                .Build();

                GetMarketplaceParticipationsResponse result =
                sellersApi.GetMarketplaceParticipations();
                Console.WriteLine(result.ToJson());
            }
            catch (LWAException e)
            {
                Console.WriteLine("LWA Exception when calling
                SellersApi#getMarketplaceParticipations");
                Console.WriteLine(e.GetErrorCode());
                Console.WriteLine(e.GetErrorMessage());
                Console.WriteLine(e.Message);
            }
            catch (ApiException e)
            {
                Console.WriteLine("Exception when calling
                SellersApi#getMarketplaceParticipations");
            }
        }
    }
}
```

```
        Console.WriteLine(e.Message);
    }
}
}
```

LWAAuthorizationCredentials instance parameters:

Name	Description	Required
ClientId	Your LWA client identifier. For more information, refer to Viewing your application information and credentials .	Yes
ClientSecret	Your LWA client secret. For more information, refer to Viewing your application information and credentials .	Yes
RefreshToken	The LWA refresh token. Get this value when the selling partner authorizes your application. For more information, refer to Authorizing Selling Partner API Applications .	No. Include RefreshToken if the operation that you call in the following step requires selling partner authorization. All operations that are not grantless operations require selling partner authorization. If you include RefreshToken , do not include Scopes.
Scopes	The scope of the LWA authorization grant. You can specify one or more Scopes values: - ScopeNotificationsAPI . For the Notifications API . - ScopeMigrationAPI .	No. Include Scopes if the operation that you call in the following step is a grantless operation . If you include Scopes , do not include the RefreshToken .
Endpoint	The LWA authentication server URI.	Yes

Example of an operation call that requires selling partner authorization:

```
C#
using Amazon.SellingPartnerAPIAA;
LWAAuthorizationCredentials lwaAuthorizationCredentials = new
LWAAuthorizationCredentials
{
    ClientId = "myClientId",
    ClientSecret = "myClientSecret",
    RefreshToken = "Aztr|...",
    Endpoint = new Uri("https://api.amazon.com/auth/o2/token")
};
```

Example of a grantless operation call:

```
C#
using Amazon.SellingPartnerAPIAA;
LWAAuthorizationCredentials lwaAuthorizationCredentials = new
LWAAuthorizationCredentials
{
    ClientId = "myClientId",
    ClientSecret = "myClientSecret",
    Scopes = new List<string>() { ScopeConstants.ScopeNotificationsAPI,
```

APP INTEGRATIONS API

App Integrations API

APPLICATION MANAGEMENT API

Application Management API

CATALOG ITEMS API

Catalog Items API

Catalog Items API Rate Limits

Catalog Items API v2022-04-01 Reference

CUSTOMER FEEDBACK API

Customer Feedback API

Customer Feedback API Rate Limits

DATA KIOSK

Data Kiosk API

Data Kiosk Schema Explorer User Guide

Data Kiosk Query Processing Finished Notification

Building Data Kiosk workflows guide

Vendor Analytics Dataset Use Case Guide

Schedule Data Kiosk Queries

DELIVERY BY AMAZON API

Delivery by Amazon API

EASY SHIP API

Easy Ship API

Tutorial: Automate your SP-API Calls Using a C# SDK

```
ScopeConstants.ScopeMigrationAPI }
Endpoint = new Uri("https://api.amazon.com/auth/o2/token")
};
```

10. To view the output of the code, you need to convert it to a console application. To do so, right-click the `SellingPartnerAPI.SellerAPI.Test` file and choose **Options**.

11. On the **Project Options** window, in the left navigation pane, expand **Build**, then choose **General**. In the **Code Generation** section, choose **Compile Target** and select **Executable with GUI**. Choose **OK**.

12. Build and run the project.

Troubleshoot a C# SDK

Restsharp errors: " IRestRequest is defined in an assembly that is not referenced. You must add a reference to assembly RestSharp Version=105.0.0.0. Culture=neutral. PublicKeyToken=598062e77f915f75 "

► [Select to expand the answer.](#)

"The type or namespace name 'Newtonsoft' could not be found (are you missing a using directive or an assembly reference?)"

► [Select to expand the answer.](#)

"Message : Could not load type of field SellingPartnerAPI.servicesApi.Client.ApiClient:rateLimiter (3) due to : Could not load file or assembly ComposableAsync.Core, Version=1.1.1.0, Culture=neutral, PublicKeyToken=null or one of its dependencies."

► [Select to expand the answer.](#)

Mapping issues with models

► [Select to expand the answer.](#)

" HttpRequestHeaders.ToJson() : no suitable method found to override."

► [Select to expand the answer.](#)

"The name BaseValidate does not exist in the current context"

► [Select to expand the answer.](#)

 Updated 18 days ago

← Tutorial: Test Selling Partner API Endpoints

Tutorial: Automate your SP-API Calls Using a Java SDK →

Did this page help you?

Yes

No

EXTERNAL FULFILLMENT

External Fulfillment Inventory API >

[Policies and Agreements](#) [Privacy notice](#) | [Conditions of use](#)

External Fulfillment Returns API >

External Fulfillment Shipping API >

External Fulfillment Errors
External Fulfillment APIs Rate Limits

FULFILLMENT BY AMAZON (FBA)

FBA Inventory API >

FBA Inventory Dynamic Sandbox Guide

FEEDS API

Feeds API >

Feed Type Values >

Feeds JSON Schemas ↗

Feeds API Best Practices
Feeds API FAQ
Feeds API Rate Limits

FINANCES

Finances API >

Transfers API >

FULFILLMENT INBOUND API

Fulfillment Inbound API >

Fulfillment Inbound API FAQ

Migrating Fulfillment Inbound workflows

Fulfillment Inbound API Rate Limits

FULFILLMENT OUTBOUND API

Fulfillment Outbound Dynamic Sandbox Guide

Fulfillment Outbound API >

Fulfillment Outbound API Rate Limits

MCF Best Practices

INVOICING

Invoices API >

Invoices API FAQ

LISTINGS

Listings Items API >

Listings Items API Rate Limits

Listings Restrictions API >

Listings Restrictions API Rate Limits

Manage Product Listings with the Selling Partner API

Building Listings Management Workflows Guide

Understanding Amazon listing status and seller-fulfilled inventory management

Listings Management Workflow Migration >

Manage Amazon Haul, Advanced Multiple-Offer, and Multiple-Fulfillment Use Cases

Listings APIs FAQ

Product Type Definitions API >

Product Type Definitions API Rate Limits

MERCHANT FULFILLMENT API

Merchant Fulfillment API >

MESSAGING API

Messaging API >

NOTIFICATIONS API

Notifications API >

Notification Type Values

Tutorial: Subscribe to the ORDER_CHANGE Notification ↗

ORDERS API

Orders API >

Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier ↗

Orders API Migration Guide

Orders API Rate Limits

PRODUCT FEES API

Product Fees API >

PRODUCT PRICING API

Product Pricing API >

- Product Pricing API and Notifications FAQ
- Price Adjustment Automation Workflows Guide
- Manage automated pricing rules with SP-API

REPLENISHMENT API

Replenishment API >

REPORTS API

Reports API >

Report Type Values >

Reports JSON Schemas ↗

Reports API FAQ

SALES API

Sales API >

SELLER WALLET API

Seller Wallet API >

Seller Wallet API Rate Limits

SELLERS API

Sellers API >

SERVICES API

Services API >

SHIPMENT INVOICING API

Shipment Invoicing API >

SHIPPING API

Shipping API v2 Resources ↗

Shipping API v1 Reference

SOLICITATIONS API

Solicitations API >

SUPPLY SOURCES API

Supply Sources API >

Multi-Location Inventory Integration Guide

Supply Sources API Rate Limits

TOKENS API

Tokens API >

UPLOADS API

Uploads API >

VEHICLES API

Vehicles API >

Vehicles API Rate Limits

VENDOR DIRECT FULFILLMENT APIS

Vendor Direct Fulfillment Dynamic Sandbox Guide

Vendor Direct Fulfillment Workflow Guide

SP-API Bill-to-Party Addresses

VENDOR DIRECT FULFILLMENT TRANSACTION STATUS API

Vendor Direct Fulfillment Transaction Status API >

VENDOR DIRECT FULFILLMENT INVENTORY API

Vendor Direct Fulfillment Inventory API >

VENDOR DIRECT FULFILLMENT ORDERS API

Vendor Direct Fulfillment Orders API >

VENDOR DIRECT FULFILLMENT SHIPPING API

Vendor Direct Fulfillment Shipping API >

VENDOR DIRECT FULFILLMENT PAYMENTS API

Vendor Direct Fulfillment Payments API >

VENDOR RETAIL PROCUREMENT INVOICES API

Vendor Retail Procurement Invoices API >

VENDOR RETAIL PROCUREMENT ORDERS API

Vendor Retail Procurement Orders API >

VENDOR RETAIL PROCUREMENT SHIPMENTS API

Vendor Retail Procurement Shipments API >

**VENDOR RETAIL PROCUREMENT TRANSACTION STATUS
API**

Vendor Retail Procurement Transaction Status API >

